

This file includes the following:

- 1) Concordance Correlation and related loss for emotion detection.
- 2) WavLM regressor for regression task instead of classification.
- 3) Custom Dataloader for Audio (WavLM) and labels for Emotion Detection.

1) Concordance Correlation Loss and MSE loss for Regression:

```
import numpy as np

from transformers import AutoFeatureExtractor, AutoModel

import torch
import torch.nn as nn
import torch.nn.functional as F

def mse_loss_fn(pred, labels, attention_mask=None, model=None):
    # Build a frame-level mask for padded batches
    # attention_mask is sample-level; convert lengths -> frame lengths
    if attention_mask is None:
        frame_mask = torch.ones_like(pred, dtype=torch.bool)
    else:
        sample_lengths = attention_mask.sum(dim=1) # (B,)
        frame_lengths = model.backbone._get_feat_extract_output_lengths(sample_lengths) # (B,)
        T = pred.size(1)
        frame_mask = torch.arange(T, device=pred.device)[None, :] < frame_lengths[:, None] # (B,T)

    loss = F.mse_loss(pred[frame_mask], labels[frame_mask])

    return loss

def ccc_value(est, target, eps = 10**-12):
    """
    Concordance correlation coefficient metric

    est: (B,T) predicted values, B=no of samples in a batch, T=no of time samples in each batch
    target: (B,T) ground truth values

    return: (B,) ccc for each sample in the batch

    """
    est_mu = torch.mean(est,axis=1)
    target_mu = torch.mean(target,axis=1)
```

```

centered_est = est - est_mu[:,None]
centered_target = target - target_mu[:,None]

cov = torch.mean( centered_est * centered_target, axis=1 )
var_est = torch.mean(centered_est**2,axis=1)
var_target = torch.mean(centered_target**2,axis=1)

ccc = (2*cov) / (var_est + var_target + (est_mu - target_mu)**2 + eps)

return ccc

def ccc_loss(est, target, eps = 10**-12):
    """
    Concordance correlation coefficient loss function

    est: (B,T) predicted values, B=no of samples in a batch, T=no of time samples in each batch
    target: (B,T) ground truth values

    return: scalar loss value
    """
    ccc = ccc_value(est, target, eps)
    loss = 1-torch.mean(ccc)

    return loss

```

2) WavLM Regressor for the above loss:

```

import numpy as np

from transformers import AutoFeatureExtractor, AutoModel

import torch
import torch.nn as nn
import torch.nn.functional as F
from transformers import WavLMModel

MODEL_NAME = "microsoft/wavlm-base"

class WavLMFrameRegressor(nn.Module):
    def __init__(self, model_name=MODEL_NAME, out_dim=1, freeze_backbone=False):
        super().__init__()
        #self.backbone = WavLMModel.from_pretrained(model_name)
        self.backbone = WavLMModel.from_pretrained(model_name, use_safetensors=True)

        hidden = self.backbone.config.hidden_size # 768 for wavlm-base
        self.head = nn.Sequential(
            #nn.Dropout(0.1),
            nn.Linear(hidden, out_dim),
        )

        if freeze_backbone:
            for p in self.backbone.parameters():

```

```

        p.requires_grad = False

    def forward(self, input_values, attention_mask=None):
        out = self.backbone(input_values=input_values, attention_mask=attention_mask)
        frames = out.last_hidden_state                # (B, T, D)
        pred = self.head(frames).squeeze(-1)         # (B, T) if out_dim=1

        return pred

def WavLMPreProcessor(model_name=MODEL_NAME):
    return AutoFeatureExtractor.from_pretrained(model_name)

```

3) Custom DataLoader for WavLM:

```

import numpy as np

from transformers import AutoFeatureExtractor, AutoModel

import torch
import torch.nn as nn
import torch.nn.functional as F
from transformers import WavLMModel

from scipy.interpolate import PchipInterpolator
from torch.utils.data import TensorDataset, DataLoader

class AudioLabelCollateFn:
    def __init__(self, batch_per_batch, chunk_size, audio_t, label_t, processor):
        self.samples = batch_per_batch
        self.audio_t = audio_t
        self.label_t = label_t
        self.processor = processor
        self.time_window = chunk_size
        self.skip_time = chunk_size+1
        self.aud_length = 16000*self.time_window #16000 -> Audio Sampling Rate
        self.skip_label_indexes = 25*self.skip_time
        fs_1 = 0.04 #Nature of data, may pass it as a argument if custom values needed
        fs_2 = 0.02
        self.t_1 = np.arange(0, self.time_window+fs_1, fs_1)
        #this is not exact. may need delay matching when evaluating
        #From history: the evaluation needs delay matching regardless
        #of how accurate t_2 is.
        self.t_2 = np.arange(2*fs_2, self.time_window+fs_2, fs_2)
        self.processor = processor

    def extract_batches(self, data, labels):
        audio_splits = []
        labels_splits = []
        #randomly extract audio chunk
        for _ in range(self.samples):
            i = np.random.randint(self.skip_label_indexes, len(labels))
            t = self.label_t[i]

```

```

        idx = np.argmin(np.abs(self.audio_t - t))
        aud_x = data[idx-self.aud_length:idx]
        audio_splits.append(aud_x)

        labl = labels[i-len(self.t_1)+1:i+1]
        cs = PchipInterpolator(self.t_1, labl)
        lab_x = cs(self.t_2)
        labels_splits.append(lab_x)

    return audio_splits, labels_splits

def __call__(self, batch):
    xs, ys = zip(*batch)

    #processor require numpy array
    xs = [x.detach().cpu().numpy() for x in xs]
    ys = [y.detach().cpu().numpy() for y in ys]

    X = []
    Y = []
    for i in range(len(xs)):
        aud_splits, label_splits = self.extract_batches(xs[i], ys[i])
        X.append(np.array(aud_splits))
        Y.append(np.array(label_splits))
    xs = np.concatenate(X, axis=0)
    ys = torch.from_numpy(np.concatenate(Y, axis=0))

    # Feature extractor pads raw audio and returns attention_mask
    inputs = self.processor(xs, sampling_rate=16000, return_tensors="pt", padding=True)

    # labels: pad to max frame length in batch
    #not necessary needed for the way I wrote the subsequent codes. But for generality
    maxT = max(y.shape[0] for y in ys)
    labels = torch.zeros(len(ys), maxT, dtype=torch.float32)
    for i, y in enumerate(ys):
        labels[i, : y.shape[0]] = y

    return inputs, labels

```